PYTHON FOR ACTUARIES · NOTEBOOK 07 · EXERCISE

# Motor Insurance Fraud Detection

Final Exercise

Build a classifier that detects fraudulent motor insurance claims. Work through three phases:

- **Phase 1**: Exploration
- **Phase 2**: Preparation and Modelling
- **Phase 3**: Evaluation

## The Dataset: `vehicle_fraud.csv`

10,050 rows of synthetic motor insurance claims:

| Column | Description |
|---|---|
| `claim_id` | Claim number |
| `policyholder_age` | Age in years |
| `occupation` | `Employed`, `Self-employed`, `Unemployed`, `Student` |
| `region` | Postcode region `PLZ_01` to `PLZ_10` |
| `schufa_score` | Credit score (300–850) |
| `customer_since_years` | Years as a customer |
| `vehicle_value` | Vehicle value |
| `vehicle_type` | `Compact`, `Mid-range`, `Luxury`, `SUV`, `Sports car` |
| `coverage_type` | `Liability`, `Partial coverage`, `Comprehensive` |
| `number_of_prior_claims` | Number of earlier claims |
| `claim_amount` | Claim amount |
| `police_report` | `1` if a police report was filed |
| `witnesses_present` | `1` if witnesses were present |
| `reporting_delay_days` | Days between accident and claim report |
| `fraud` | Target: `1` = fraudulent, `0` = legitimate |

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix, precision_recall_curve)
from sklearn.inspection import permutation_importance

# Seaborn style for all plots
sns.set_theme(style='whitegrid')

# Avoid truncation of DataFrame output in Jupyter
pd.set_option('display.expand_frame_repr', False)

# Display all columns in the DataFrame output
pd.set_option('display.max_columns', None)

# Define a separator for better readability
sep = "=" * 50
```

```
# Load the dataset
df = pd.read_csv('vehicle_fraud.csv')
df.head()
```

| | claim_id | policyholder_age | occupation | region | schufa_score | customer_since_years | vehicle_value | vehicle_type | coverage_type | number_of_prior_claims | claim_amount | police_report | witnesses_present | reporting_delay_days | fraud |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 108777 | 38 | Unemployed | PLZ_05 | 575.0 | 14 | 75699.57 | Luxury | Liability | 2 | 15055.72 | 0 | 0 | 31 | 0 |
| 1 | 102200 | 42 | Employed | PLZ_10 | 653.0 | 21 | 70254.54 | Sports car | Partial coverage | 2 | 4751.62 | 1 | 0 | 16 | 0 |
| 2 | 109597 | 27 | Self-employed | PLZ_09 | 525.0 | 2 | 9686.20 | Compact | Comprehensive | 2 | 7600.43 | 0 | 0 | 44 | 0 |
| 3 | 108491 | 43 | Employed | PLZ_03 | 619.0 | 14 | 77918.56 | Compact | Comprehensive | 2 | 7524.33 | 0 | 1 | 31 | 0 |
| 4 | 103315 | 45 | Student | PLZ_04 | 624.0 | 8 | 3660.94 | Luxury | Partial coverage | 4 | 25664.09 | 0 | 0 | 33 | 1 |

# Phase 1: Exploration

Get to know the data: size, missing values, fraud rate, distributions and the first fraud patterns.

## 1.1 Overview

**Task 1:** How many rows and columns does the dataset have? Which columns are numeric, which are text?

```
# Task 1: Shape and data types
# Hint: df.shape, df.info()
```

**Task 2:** Are there missing values?

```
# Task 2: Missing values
# Hint: df.isna().sum()
```

**Task 3:** What is the fraud rate?

```
# Task 3: Fraud rate
# Hint: df['fraud'].value_counts(normalize=True)
```

### 1.2 Distributions

**Task 1:** Plot a histogram of `claim_amount`. Is it symmetric or skewed?

```
# Task 1: Histogram of claim_amount
# Hint: df['claim_amount'].plot(kind='hist', bins=50)
#       plt.title('...'), plt.xlabel('...'), plt.ylabel('...')
#       plt.show()
```

**Task 2:** Compare `claim_amount` for fraudulent and legitimate claims with a box plot.

```
# Task 2: Box plot of claim_amount by fraud
# Hint: sns.boxplot(data=df, x='fraud', y='claim_amount')
```

### 1.3 Fraud Patterns

**Task 1:** Compare the mean of every numeric column for `fraud == 0` and `fraud == 1`.

```
# Task 1: Means by fraud status
# Hint: df.groupby('fraud').mean(numeric_only=True)
```

**Task 2:** Compute the correlation of every numeric column with `fraud` and sort it.

```
# Task 2: Correlation with fraud
# Hint: df.corr(numeric_only=True)['fraud'].sort_values()
```

**Task 3:** Compute the fraud rate for each combination of `police_report` and `witnesses_present`.

```
# Task 3: Fraud rate by documentation
# Hint: df.groupby(['police_report', 'witnesses_present'])['fraud'].mean()
```

**Task 4:** Compute the fraud rate for new customers (`customer_since_years < 3`) with many prior claims (`number_of_prior_claims >= 3`).

```
# Task 4: Fraud rate of new customers with many prior claims
# Hint: new_many = (df['customer_since_years'] < 3) & (df['number_of_prior_claims'] >= 3)
#       df.groupby(new_many)['fraud'].mean()
```

## Phase 2: Preparation and Modelling

Apply Notebooks 05 and 06: clean, engineer features, encode, split, fill, scale and train one or more models.

## 2.1 Cleaning

**Task 1:** How many rows are duplicates? Remove them.

```
# Task 1: Duplicates
# Hint: df.duplicated().sum()
#       df = df.drop_duplicates()
```

**Task 2:** Find the impossible ages with `describe()` and replace them with `np.nan`.

```
# Task 2: Impossible ages
# Hint: df['policyholder_age'].describe()
#       df.loc[df['policyholder_age'] < 18, 'policyholder_age'] = np.nan
```

**Task 3:** Add the indicator column `schufa_missing`. Compare the fraud rate of claims with and without `schufa_score`.

```
# Task 3: Indicator column and fraud rate
# Hint: df['schufa_missing'] = df['schufa_score'].isna().astype(int)
#       df.groupby('schufa_missing')['fraud'].mean()
```

## 2.2 Feature Engineering

Create four new columns:

**Task 1:** `claim_value_ratio` = `claim_amount` / `vehicle_value`

```
# Task 1: Claim-to-value ratio
# Hint: df['claim_value_ratio'] = df['claim_amount'] / df['vehicle_value']
```

**Task 2:** `no_report_no_witness` = `1` if there is neither a police report nor witnesses

```
# Task 2: Neither police report nor witnesses
# Hint: ((df['police_report'] == 0) & (df['witnesses_present'] == 0)).astype(int)
```

**Task 3:** `new_customer_many_claims` = `1` for customers with `customer_since_years < 3` and `number_of_prior_claims >= 3`

```
# Task 3: New customer with many prior claims
```

**Task 4:** `log_claim_amount` = `np.log(claim_amount)` for the skewed claim amounts

```
# Task 4: Log-transformed claim amount
```

Then compare the mean `claim_value_ratio` for fraudulent and legitimate claims.

```
# Mean claim_value_ratio by fraud status
# Hint: df.groupby('fraud')['claim_value_ratio'].mean()
```

## 2.3 Encoding, Split, Missing Values and Scaling

**Task 1:** One-hot encode `occupation`, `region`, `vehicle_type` and `coverage_type` with `pd.get_dummies()`. Store the result in `df_encoded`.

```
# Task 1: One-hot encoding
# Hint: pd.get_dummies(df, columns=[...], drop_first=True, dtype=int)
```

**Task 2:** Split into features `X` (without `claim_id` and `fraud`) and target `y`.

```
# Task 2: Features and target
# Hint: X = df_encoded.drop(columns=['claim_id', 'fraud'])
#       y = df_encoded['fraud']
```

**Task 3:** Split into 80% training and 20% test data. `stratify=y` keeps the fraud rate equal in both sets.

```
# Task 3: Train/test split
# Hint: train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

**Task 4:** Fill the missing values in both sets with the medians of the training data.

```
# Task 4: Fill missing values with the training medians
# Hint: medians = X_train.median()
#       X_train = X_train.fillna(medians)
#       X_test = X_test.fillna(medians)
```

**Task 5:** Scale the features with `StandardScaler`. Fit the scaler on the training data only.

```
# Task 5: Scaling
# Hint: scaler = StandardScaler()
#       X_train_scaled = scaler.fit_transform(X_train)
#       X_test_scaled = scaler.transform(X_test)
```

## 2.4 Training

Train one or more of the following classifiers.

With only 12% fraud, a model mostly learns the legitimate claims. Class weighting counteracts this: each fraud case counts as much as about 7.5 legitimate claims (number of legitimate / number of fraudulent claims), so both classes carry the same total weight. `class_weight='balanced'` and `scale_pos_weight` do the same thing; `XGBClassifier` just calls it differently. `MLPClassifier` has no class weighting.

| Model | Input | Class weighting |
|---|---|---|
| `LogisticRegression(max_iter=1000)` | scaled | `class_weight='balanced'` |
| `RandomForestClassifier(n_estimators=100)` | unscaled | `class_weight='balanced'` |
| `XGBClassifier(n_estimators=100)` | unscaled | `scale_pos_weight` = number of legitimate / number of fraudulent claims |
| `MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500, early_stopping=True)` | scaled | none |

Use `random_state=42` for every model.

```
# Train one or more models
# Hint: lr_model = LogisticRegression(max_iter=1000, class_weight='balanced',
    random_state=42)
#       lr_model.fit(X_train_scaled, y_train)
# Hint: for XGBoost, scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
```

## Phase 3: Evaluation

Compare the models with metrics, find the important features and pick the cheapest threshold.

### 3.1 Metrics

| Metric | Question |
|---|---|
| Precision | Of the flagged claims, how many are fraud? |
| Recall | Of all fraud cases, how many are flagged? |
| F1-score | Balance of precision and recall |
| ROC-AUC | How well does the model rank fraud above non-fraud? |

Store your trained models with their test data in the dictionary `models`. Keep only the entries of the models you trained.

```
# Uncomment the entries of the models you trained
models = {
    # 'Logistic Regression': (lr_model, X_test_scaled),
    # 'Random Forest': (rf_model, X_test),
    # 'XGBoost': (xgb_model, X_test),
    # 'Neural Network': (mlp_model, X_test_scaled),
}
```

**Task 1:** For each model, compute accuracy, precision, recall, F1-score and ROC-AUC. Show the results as a DataFrame.

```
# Task 1: Metrics table
# Hint: results = {}
#       for name, (model, X_eval) in models.items():
#           y_pred = model.predict(X_eval)
#           y_proba = model.predict_proba(X_eval)[:, 1]
#           results[name] = {'Accuracy': accuracy_score(y_test, y_pred), ...,
#                            'ROC-AUC': roc_auc_score(y_test, y_proba)}
#       results_df = pd.DataFrame(results).T
```

**Task 2:** Plot a confusion matrix for each model.

```
# Task 2: Confusion matrices
# Hint: for name, (model, X_eval) in models.items():
#           cm = confusion_matrix(y_test, model.predict(X_eval))
#           sns.heatmap(data=cm, annot=True, fmt='d', cmap='Blues')
#           plt.title(name), plt.xlabel('Predicted'), plt.ylabel('Actual')
#           plt.show()
#       fmt='d' shows whole numbers
```

### 3.2 Feature Importance

`permutation_importance()` shuffles one feature at a time in the test data and measures how much the ROC-AUC drops. A large drop means the model relies on that feature.

**Task 1:** Pick one model from `models` and compute its permutation importance.

```
# Task 1: Permutation importance
# Hint: name = 'Random Forest'
#       model, X_eval = models[name]
#       perm = permutation_importance(model, X_eval, y_test, n_repeats=10,
#                                     scoring='roc_auc', random_state=42)
```

**Task 2:** Plot the ten most important features as a horizontal bar chart.

```
# Task 2: Top 10 features
# Hint: importances = pd.Series(perm.importances_mean, index=X_test.columns)
#       top10 = importances.sort_values().tail(10)
#       top10.plot(kind='barh')
```

**Task 3:** Do your new columns from 2.1 and 2.2 appear among them?

### 3.3 Threshold Tuning

`predict()` flags a claim as fraud if its predicted probability is at least 0.5. A lower threshold catches more fraud (higher recall) but causes more false alarms (lower precision). The best threshold depends on the costs:

- Every flagged claim is investigated. An investigation costs 500 €.
- Every missed fraud case is paid out. It costs its `claim_amount`.

$$\text{Cost}(t) = 500 \cdot \text{number of flagged claims} + \text{sum of claim amounts of missed fraud cases}$$

**Task 1:** For the model from 3.2, compute precision and recall for every threshold with `precision_recall_curve()` and plot recall against precision.

```
# Task 1: Precision-recall curve
# Hint: y_proba = model.predict_proba(X_eval)[:, 1]
#       precisions, recalls, thresholds = precision_recall_curve(y_test, y_proba)
#       pr_curve = pd.Series(precisions, index=recalls)
#       pr_curve.plot(kind='line')
```

**Task 2:** Compute the total cost for the thresholds 0.01, 0.02, ..., 0.99 and plot cost against threshold. Which threshold has the lowest cost? Compare it with 0.5.

```
# Task 2: Total cost per threshold
# Hint: def cost_table(investigation_cost):
#           rows = []
#           for t in np.arange(1, 100) / 100:
#               flagged = y_proba >= t
#               missed = ~flagged & (y_test == 1)
#               cost = investigation_cost * flagged.sum() + X_test.loc[missed,
    'claim_amount'].sum()
#               rows.append({'threshold': t, 'flagged': flagged.sum(),
#                            'recall': recall_score(y_test, flagged), 'cost': cost})
#           return pd.DataFrame(rows)
# Hint: cost_curve = costs.set_index('threshold')['cost']
#       cost_curve.plot(kind='line')
#       costs.loc[costs['cost'].idxmin()]
```

**Task 3:** Repeat Task 2 with investigation costs of 200 € and 3000 €. How does the best threshold move?

```
# Task 3: Investigation costs of 200 € and 3000 €
```

## Summary

- **Exploration:** fraud rate, skewed distributions, fraud rates by group
- **Preparation and modelling:** duplicates, impossible values, indicator for missing values, feature engineering, one-hot encoding, stratified split, filling missing values, scaling, class weights
- **Evaluation:** precision, recall, F1-score, ROC-AUC, confusion matrix, permutation importance, cost-based threshold tuning
- scikit-learn User Guide
- XGBoost Documentation